

EMPIRICAL STUDY

Multi-view coding-style transitions for leakage-safe paired detection of LLM-paraphrased code

Jin Huang¹  | Qiao Li¹ | Qisen Gao²¹Guangxi Vocational Normal University, Nanning 530007, Guangxi Zhuang Autonomous Region, China | ²Gansu University of Political Science and Law, Lanzhou 730070, Gansu Province, China |**Correspondence:** Jin Huang (614938561@qq.com). Co-author emails: Qiao Li (1339275715@qq.com); Qisen Gao (1350728839@qq.com).**Funding:** No funding was received for this study.**Keywords:** code paraphrasing | paired provenance | code stylometry | leakage-safe evaluation | distribution shift | interpretable representation

ABSTRACT

Detecting paraphrased code requires distinguishing a source–candidate relationship from resemblance to machine-written code. We study this paired provenance problem using multi-view style-transition features (MSTF). The representation combines 28 descriptors of each endpoint with signed and source-normalised differences, producing 112 features for XGBoost. Evaluation isolates both human origins and exact code content before constructing common train–test pairs. Under matched XGBoost, adding signed differences to the inherited endpoint features increases strict-clean mean F1 from 0.9149 to 0.9317 (+1.683 percentage points). Full MSTF exceeds the retained LPcode-original multilayer perceptron by 7.563, 11.160 and 7.141 points under held-out generators, combined deterministic transformations and held-out languages, respectively. The language result is descriptive three-seed evidence. Fixed-classifier ablations support contributions from both enhanced endpoint descriptors and signed differences. The additional relative-difference block provides negligible incremental benefit. Hard-negative pairing reduces absolute performance but preserves the signed-transition advantage. These findings support an explicit transition representation within the evaluated LPcode corpus, four generators, four languages and specified transformations. They do not establish general-purpose AI-code detection or individual authorship.

1 | Introduction

Large language models can rewrite code while retaining much of its intended functionality. Rewriting may alter identifiers, comments, layout and structural choices without creating an entirely new solution. Such changes complicate provenance analysis because the candidate can remain recognisably related to an earlier human source. The relevant question is then relational: does this particular candidate exhibit the transition associated with paraphrasing this particular source?

LPcode establishes a direct predecessor for this paired detection setting [1]. Candidate-only detection and paired detection should nevertheless remain conceptually separate. A candidate-only classifier estimates whether a program resembles its training examples of machine-written code. A paired classifier can additionally use the observed source and compare the endpoints. This

distinction matters when stylistic characteristics vary across developers, programming languages and generators. A naming pattern may be common in both human and machine code, whereas its change relative to a supplied source may be informative. Pairing alone does not guarantee a credible evaluation. A negative example can connect a source to a candidate derived from another human origin. Splitting only on the first source leaves that second origin available in another partition. Different origin identifiers can also contain identical code, creating another path for train–test overlap. An evaluation must therefore isolate both origins and exact content, then construct pairs within the resulting partitions. General methodological warnings about machine-learning evaluation motivate treating these constraints as part of the experimental design, not post-processing [2].

We study multi-view style-transition features, abbreviated MSTF, under this leakage-safe protocol. MSTF combines richer endpoint descriptors with explicit signed differences and a relative-difference block. The signed difference is a deterministic re-expression of the endpoint values, not additional information once both endpoints are known. Its motivation concerns learnability: explicit changes may be easier for a finite classifier to use than differences reconstructed implicitly from concatenated endpoints.

The evaluation separates a controlled representation question from a whole-system comparison. In strict clean data, signed differences improve a matched XGBoost endpoint baseline by 1.683 percentage points. Full MSTF shows larger advantages against the retained LPCODE-original classifier under the evaluated distribution shifts. Those larger differences include changes in descriptors and classifier, so they do not isolate a transition effect. Controlled ablations provide that separation and retain the near-null relative-difference result.

The study makes four contributions. First, it operationalises paired style change using explicit, reproducible endpoint and transition descriptors. Second, it specifies exact-content and dual-endpoint isolation with shared pairs across compared methods. Third, it evaluates this design under generator, deterministic transformation and language shifts. Fourth, fixed-classifier ablations and negative-pair sensitivity distinguish representation contributions from descriptive feature rankings. The objective is a bounded representation and protocol study, not a claim of unrestricted detection.

2 | Background and related work

2.1 | Generated-code and paraphrased-code detection

Park and colleagues use coding-style features for detecting LLM-paraphrased code and identifying the responsible generator [1]. Their data and paired formulation are inherited here; neither is claimed as a new contribution. CodeMirage broadens benchmarking to multilingual generated and paraphrased source code from production-level models [3]. It is an important independent evaluation resource, but this study does not report results on that benchmark.

Other studies examine detection at different program granularities and with multilingual stylistic representations. Rahman and colleagues compare detectors across function and class granularities [4]. Gurioli and colleagues investigate AI-written programs using multilingual code stylometry [5]. These studies establish the relevance of code-specific evaluation. Their tasks and data should not be treated as directly matched competitors to the present source-candidate experiment. Multilingual coverage also differs from withholding an entire language during fitting.

Natural-language detection offers related ideas but not interchangeable evidence. DetectGPT evaluates probability curvature, while Fast-DetectGPT uses conditional probability curvature for efficient detection [6, 7]. Work on reliable text detection highlights limitations under changes to generated content and distributional assumptions [8]. These studies motivate explicit robustness boundaries. Their results do not establish code detection performance, and their probability-based access requirements differ from handcrafted paired features.

2.2 | Code stylometry and representation learning

Code stylometry uses lexical, syntactic and layout regularities as predictive measurements. Caliskan-Islam and colleagues study programmer de-anonymisation using code stylometry [9]. Bogomolov and colleagues develop language-agnostic source-code authorship attribution and discuss software-engineering applicability [10]. These studies explain why style measurements can be informative. However, an authorship label identifies a programmer, whereas our label describes a constructed source-candidate provenance relationship.

Learned code representations offer richer alternatives to handcrafted measurements. CodeBERT jointly represents programming and natural languages [11]. GraphCodeBERT incorporates data flow [12], CodeT5 uses identifier-aware pre-training [13], and UniXcoder studies cross-modal code representation [14]. CodeSearchNet evaluates semantic code search [15]. These resources demonstrate that code representation extends beyond style. None is evaluated as a new baseline in the frozen experiments, and MSTF is not claimed to outperform them.

The choice of a compact descriptor vector serves experimental control. Each coordinate has an explicit extraction rule, and feature families can be changed while retaining the same classifier and pairs. This transparency comes with a limitation: handcrafted style cannot fully represent program behaviour or semantic equivalence. A stronger semantic representation could resolve ambiguities that MSTF cannot observe.

2.3 | Robust attribution and evaluation under shift

Quiring and colleagues show that source-code authorship attribution can be misled by semantics-preserving transformations guided by an adversarial search [16]. RoPGen studies robust attribution through automatic coding-style transformation [17]. These are stronger adversarial settings than the fixed transformations evaluated here. Their findings motivate perturbation tests while cautioning against interpreting a successful deterministic test as general adversarial robustness.

Evaluation design is equally important. Arp and colleagues describe pitfalls that can produce unrealistic conclusions in security-related machine learning [2]. Here, leakage can arise through either endpoint and through duplicate content. We therefore specify the unit of isolation and reuse identical pair manifests across representations. This protocol addresses identified overlap channels; it does not eliminate all possible dataset artefacts.

The closest relationship is to LPCODE, not to generic text detection or programmer identification. The extension combines enhanced descriptors, explicit differences and a stricter paired evaluation. The comparison matrix in the accompanying reference audit distinguishes these axes without assigning undocumented deficiencies to prior studies. A capability marked as unestablished means it was not verified under our exact protocol, rather than absent from the original work.

3 | Problem formulation and motivation

3.1 | A conditional provenance task

Let h denote a human source program, c a candidate, and y the dataset relationship label. The classifier estimates $P(y = 1 | h, c)$. A positive pair connects a source to its recorded LLM paraphrase. A negative pair connects the source to a non-matching candidate from a different exact-content component. Importantly, that candidate may itself be LLM-generated. The negative class therefore means non-matching provenance, not necessarily human-written code.

This construction defines what the model can be evaluated to recognise. It does not provide a semantic proof that two programs are equivalent or that one was historically derived from the other. Labels come from the corpus generation relationship and the registered negative construction. A deployment that supplies arbitrary source candidates or naturally occurring unrelated edits may have a different negative distribution.

3.2 | Why explicitly represent change?

Write $F_h = F(h)$ and $F_c = F(c)$ for the endpoint descriptors. A candidate-only representation omits the supplied source; endpoint concatenation $[F_h; F_c]$ retains both measurements. Explicit transition features expose direction and magnitude relative to the source. For example, the same candidate comment ratio can correspond to an increase for one source and a decrease for another. This example is a modelling illustration, not an observed error case.

Because subtraction is deterministic, explicit differences do not enlarge the information available to an unconstrained classifier given both endpoints. They change its coordinates and inductive bias. Axis-aligned tree splits may exploit a precomputed difference more directly than two separate endpoint coordinates. Whether that re-expression helps is an empirical question answered by matched ablation, not by the definition itself.

3.3 | Scope of the hypothesis

The working hypothesis is that explicit changes and richer endpoints jointly improve the evaluated paired classification. It does not require every transition block to help. Source-scale normalisation might be useful when feature magnitudes vary, but it can also amplify near-zero denominators or duplicate signed-difference information. The relative block remains in the full specification so that its weak incremental result is visible and reproducible.

4 | Method

4.1 | Endpoint measurements

The extractor produces 28 measurements for each endpoint. Ten reproduce the inherited Lpcode style implementation, and eighteen add lexical, structural and formatting views. Table 1 lists every measurement in extraction order. The same implementation processes the human and candidate endpoints. Inherited measurements retain their original heuristics rather than being silently replaced with improved parsers.

4.2 | Enhanced feature definitions and parsing

Identifier entropy is Shannon entropy in bits over identifier occurrences, not unique spellings. Identifier length statistics use those occurrences. Keyword, operator and literal densities divide by effective lexical items, excluding punctuation and comments. Quoted strings and character literals each count as one literal; their contents are not tokenised as code.

Structural measurements use Python’s abstract syntax tree or pinned Tree-sitter grammars for C, C++ and Java. Branch, loop, return and exception densities divide by structural statement units. Statement density divides those units by non-blank physical lines. AST depth counts named structural edges from the root. Cyclomatic complexity is an implementation-specific count of one plus decision points, with zero for content lacking effective tokens or structure. These rules are proxies, not a language-independent formal complexity measure.

Blank-line ratio uses all physical lines. Line-length mean and population standard deviation use non-blank lines. Indentation entropy uses their leading whitespace widths, expanding tabs to four-column stops. Parse failures preserve lexical and formatting measurements and substitute conservative token-based structural estimates. Empty input produces zero enhanced measurements. Finite-value checks reject invalid vectors rather than silently propagating missing values.

4.3 | Signed and relative transitions

The signed transition is computed coordinate-wise:

$$\Delta F = F_c - F_h. \quad (1)$$

Its sign distinguishes increases from decreases, and its magnitude measures change in the original feature units. The source-normalised relative transition is:

$$R_j = \frac{F_{c,j} - F_{h,j}}{|F_{h,j}| + \epsilon}, \quad \epsilon = 10^{-8}. \quad (2)$$

The stabilising constant is inside the denominator, consistent with the implementation. This avoids division by zero but does not bound the relative magnitude. Relative values should therefore not be interpreted as stable percentage changes when the source measurement is near zero.

4.4 | Full representation and classifier

The complete multi-view representation concatenates four blocks:

$$\text{MSTF}(h, c) = [F_h; F_c; \Delta F; R] \in \mathbb{R}^{112}. \quad (3)$$

Figure 1 shows the construction without implying that every block is necessary. Endpoint blocks retain measured style levels, while difference blocks make source-relative change explicit. A4 omits the relative block and has 84 dimensions; A5 is the registered full 112-dimensional MSTF.

All controlled A0–A5 variants use the same XGBoost configuration [18]. It has 300 trees, maximum depth four, learning rate 0.05, row and column subsampling 0.8, and log-loss evaluation. Fitting uses one CPU thread and the registered seed. A StandardScaler is fitted on training features only and applied to

TABLE 1 | Endpoint feature inventory. Indices are one-based positions in the 28-dimensional vector. Explanations are modelling motivations, not measured causal effects.

Indices / family	Exact feature names in order	Motivation
1–10 / Inherited style	function_naming_consistency; variable_naming_consistency; class_naming_consistency; constant_naming_consistency; indentation_consistency; avg_function_length; avg_nesting_depth; comment_ratio; avg_function_name_length; avg_variable_name_length	Naming consistency for functions, variables, classes and constants; indentation consistency; function length; nesting depth; comment ratio; function and variable name lengths. Measures retained stylistic conventions.
11–16 / Lexical	identifier_entropy; identifier_length_mean; identifier_length_std; keyword_density; operator_density; literal_density	Occurrence entropy and length statistics, plus lexical densities. Describes renaming and token-composition changes.
17–24 / Structural / syntax	ast_depth; branch_density; loop_density; function_count; statement_density; cyclomatic_complexity; return_density; exception_density	Depth, control-flow and statement proxies. Describes changes to organisation beyond surface naming.
25–28 / Formatting / layout	blank_line_ratio; line_length_mean; line_length_std; indentation_entropy	Whitespace and line-length statistics. Describes layout rewriting and normalisation.

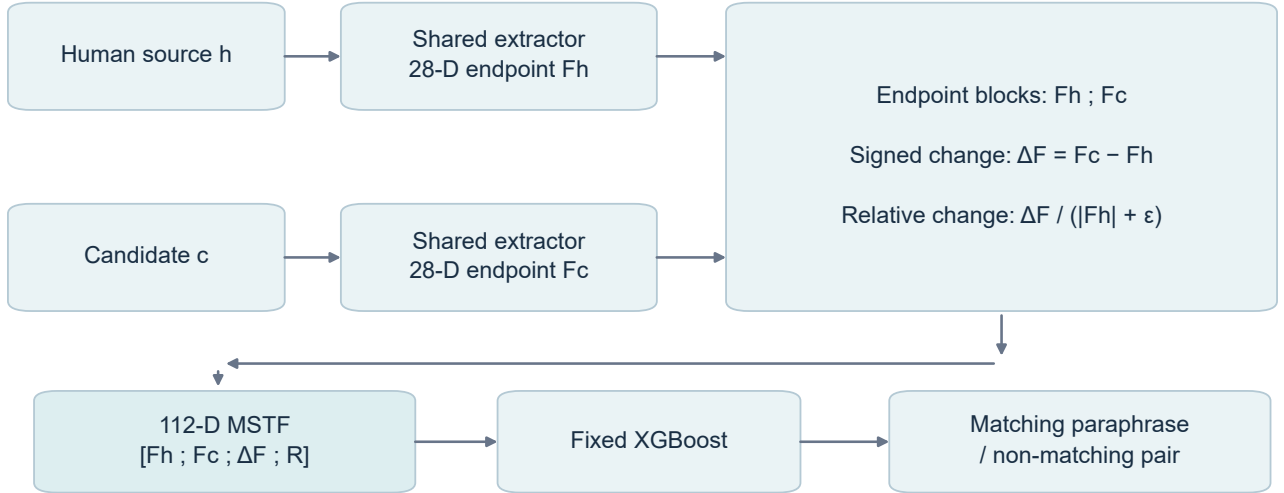


FIGURE 1 | MSTF architecture. A shared extractor yields 28 measurements per endpoint. Signed and relative differences are deterministic functions of these measurements. Concatenation yields 112 inputs to the fitted classifier; the output concerns the source–candidate relationship.

held-out features. The retained LPcode-original comparator instead uses the inherited ten descriptors, endpoint concatenation and scikit-learn’s MLPClassifier defaults [19]. Consequently, full-MSTF versus original-MLP comparisons are system comparisons, not single-factor ablations.

5 | Leakage-safe evaluation protocol

5.1 | Hidden second origins and exact-content components

A pair has two provenance endpoints, even when its storage record names only one source. Suppose a negative pair contains human source h_i and a candidate paraphrased from h_j . Assigning the record by i alone does not prevent the origin j from entering another partition. This hidden second-origin channel can contaminate a naive paired evaluation.

Exact-content isolation addresses a different channel. Origin identifiers are grouped through shared exact code hashes into connected equivalence components. A component, rather than an individual record, becomes the partition unit. Identical endpoint content therefore cannot cross train and test simply because it carries another identifier. Hash equality detects exact content,

not approximate semantic clones; the latter remain a validity threat.

5.2 | Partition first, construct pairs second

Positive-bank origins are partitioned before negative pairs are constructed. Both origins of every resulting pair belong to the same partition. Negatives use cross-component derangement within the permitted language and generator pools, retaining one negative for each positive. Candidate and source identities, component membership, class balance and exact-code overlap are checked explicitly. This order prevents negative construction from reconnecting partitions after the split.

Compared methods reuse the same pair objects and train–test index hashes. Feature families and classifiers do not receive separately sampled test sets. This common-pair design enables paired differences and prevents representation comparisons from inheriting avoidable sampling differences. Figure 2 summarises the isolation constraints and the four evaluation environments.

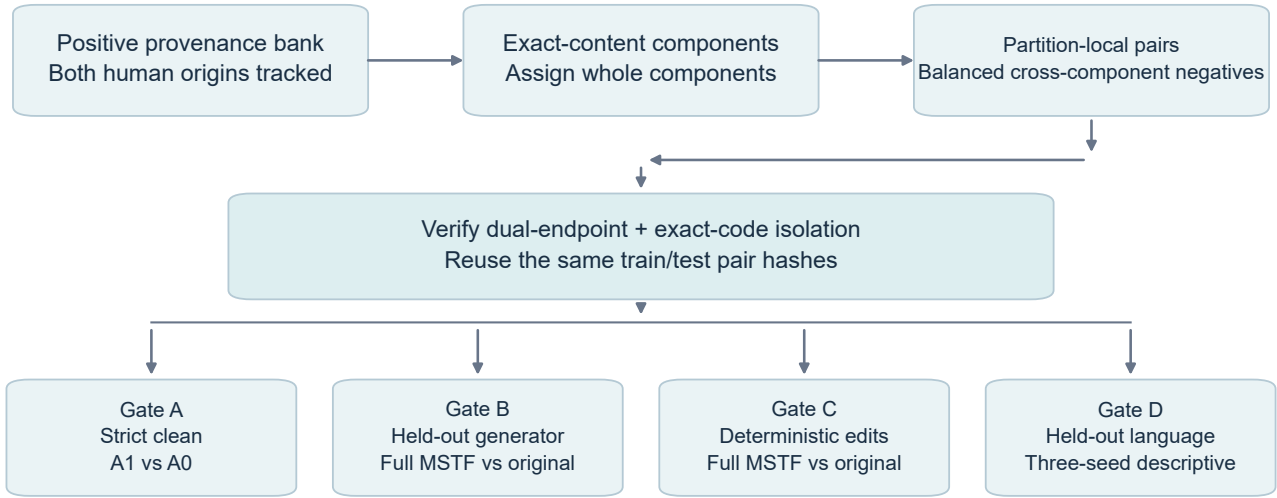


FIGURE 2 | Leakage-safe evaluation. Exact-content components are assigned before balanced pairs are constructed within each partition. Both endpoint origins and exact code hashes must remain disjoint. Common pair manifests are reused across methods. The four gates test separate shifts rather than pooling them into one test set.

5.3 | Held-out generators

Gate B excludes one generator from training and tests its candidates on held-out origins. Training candidates come from the other three generators. Source-origin partitions and exact-content constraints still apply; generator exclusion does not replace origin isolation. The protocol is repeated for GPT-3.5, Gemini-Pro, WizardCoder-33B and DeepSeek-Coder-33B-Instruct. Per-generator summaries retain language balance rather than allowing the largest language subset to determine the result.

5.4 | Deterministic style transformations

Gate C fits models on untransformed training pairs and transforms only the candidate endpoint of the test pairs. Conditions comprise comment removal, identifier renaming, formatting normalisation, comment injection and a combined transformation. The combined sequence applies comment removal, identifier renaming and formatting normalisation, in that order. Comment injection is a separate condition and is not part of the combined sequence.

Transformation statistics distinguish changed snippets, successful transformation execution and parsing regression. The recorded attack-success field denotes transformation execution, not successful detector evasion. No parsing regression is recorded in the audited conditions, but syntactic validity does not prove semantic preservation. The experiment evaluates these deterministic edits only, without adaptive search against classifier predictions.

5.5 | Held-out languages

Gate D trains on three complete language banks and evaluates the fourth language, rotating over C, C++, Java and Python. Its banks are reconstructed from the strict pair protocol, with exact-code isolation across train and test. This is not ordinary multilingual

cross-validation: the test language is absent from classifier fitting. There are three seeded fits per language and method, rather than five test folds within each seed.

5.6 | Uncertainty and dependence

The registered seeds are 42, 123 and 2024. Gates A–C use five folds per seed. Uncertainty intervals resample seed clusters with replacement, retaining the folds, languages and relevant held-out conditions inside each selected cluster. The stored bootstrap uses 10,000 replicates and bootstrap seed 250217749. Paired differences are formed on matching evaluation units before aggregation.

Only three seed clusters are available. Repeated bootstrap draws do not create additional independent experiments, and narrow intervals do not imply broad population-level certainty. Overlapping cross-validation training sets also make fold-level independence inappropriate [20]. We report the stored 95% CI ranges as limited seed-cluster summaries, not as significance tests. Gate D is explicitly descriptive, and no multiplicity-adjusted or unadjusted hypothesis-testing claim is made.

6 | Experimental setup

6.1 | Corpus and evidence units

The experiments use the LPcode corpus and four programming languages described above [1]. The relevant generators are the four recorded model sources, not an open-ended sample of current LLMs. The formal ledgers contain 480 Gate-A, 960 Gate-B, 1,440 Gate-C and 48 Gate-D method-level evaluation records. These counts describe experimental cells, not independent programs or independent datasets. Exact pair counts and class counts vary by partition and are retained in the source manifests.

The fixed-classifier ablation contains 360 clean and 1,440 held-out-generator records. Each clean variant has 60 language–seed–fold cells; each held-out variant has 240 language–generator–seed–fold cells. The mechanism analysis reuses reconstructed saved splits and is descriptive. Neither auxiliary records nor non-formal outputs are substituted for the frozen gate ledgers.

6.2 | Baselines and controlled variants

Three identities require explicit separation. LPcode original means ten endpoint measurements concatenated for an MLP. The matched XGBoost baseline, A0, uses the same ten measurements but a different classifier. A1 adds signed differences to A0. Full MSTF, A5, uses enhanced measurements and all four blocks with XGBoost. Intermediate variants in Table 3 isolate representation changes while keeping the model configuration fixed.

The strict-clean headline concerns A1 versus A0. The held-out-generator, combined-transformation and held-out-language headlines concern A5 versus the retained original MLP. A controlled A5 versus A0 comparison is reported separately in the ablation. These identities prevent the larger cross-system differences from being interpreted as evidence for subtraction alone.

6.3 | Metrics and aggregation

For each test cell, F1 is the positive-class harmonic mean:

$$F_1 = \frac{2TP}{2TP + FP + FN}. \quad (4)$$

The implementation uses scikit-learn’s binary F1 with positive label one and zero-division handling set to zero. It does not average F1 over the two class labels. Throughout this manuscript, mean F1 denotes equal-weight aggregation over the configured experimental cells. Languages and, where applicable, held-out generators receive equal weight. This clarifies the older artefact label “macro”, which denotes aggregation across evaluation strata rather than class-macro F1.

Differences are expressed in percentage points, calculated as 100 times the paired F1 difference. Precision, recall, AUROC and Matthews correlation are retained as secondary outcomes in the evidence files. Fold standard deviations describe dispersion among correlated experimental cells and are not standard errors. Table 6 instead reports standard deviation over the three seeds for each held-out language.

6.4 | Implementation and reproducibility

The pinned evaluation environment specifies Python 3.11 or later, NumPy 1.23.5, scikit-learn 1.2.0 and XGBoost 2.1.4. Tree-sitter 0.26.0 uses C 0.24.2, C++ 0.23.4 and Java 0.23.5 grammar wheels. Feature extraction requires no grammar downloads at runtime. Cache digests, pair hashes, configuration hashes and result manifests link the reported metrics to the registered inputs.

All numbers and quantitative figure labels in this revision are regenerated from saved evidence, without refitting models. Original manuscripts and frozen gate files are preserved. The local evidence archive supports audit, but public reproducibility additionally requires a populated, licensed and versioned repository.

The current release limitation is stated in Data availability rather than replaced by an unsupported claim of access.

7 | Results

7.1 | Relationship to the original LPcode implementation

The inherited ten-feature implementation and original MLP provide continuity with LPcode. The present strict-origin evaluation changes the partition and pairing protocol, so its scores are not direct replications of published LPcode scores. The formal evidence table identifies XGBoost with concatenation plus signed difference as the selected strict-origin transition candidate. We do not treat that label as proof of reproducing every original experimental claim.

Table 2 gives absolute scores and the exact comparison for each formal gate. Figure 3 displays the paired differences using the stored uncertainty summaries. The clean gate supplies controlled evidence for signed differences; the remaining gates test the complete system against the retained original comparator.

7.2 | Signed differences improve the strict-clean matched comparator

Adding signed differences increases mean F1 from 0.9149 to 0.9317, a 1.683-point advantage with interval [1.466, 1.932] points. The classifier, official endpoint descriptors and test pairs are matched. Each language has a positive mean difference. The effect is modest in absolute magnitude, but it supplies direct evidence that explicit subtraction can benefit the fixed learner beyond endpoint concatenation.

This result concerns the 30-dimensional A1 representation, not full 112-dimensional MSTF. It supports a representation-level contribution within the measured setting. It does not show that source-relative differences contain information unavailable in the concatenated endpoints, nor that they outperform other possible reparameterisations.

7.3 | Full MSTF retains performance under generator exclusion

Across held-out generators, full MSTF reaches 0.9671 mean F1 against 0.8915 for LPcode original. The paired advantage is 7.563 points, with interval [7.452, 7.627] points. Table 4 shows positive differences for every held-out generator. The absolute scores are necessary because a large difference can reflect a weaker comparator as well as a strong proposed method.

Generator exclusion leaves many other properties of the corpus shared. The result therefore supports transfer among these recorded generators, not transfer to arbitrary future models. The fixed-XGBoost ablation below separates part of the representation contribution from the classifier change in this system-level comparison.

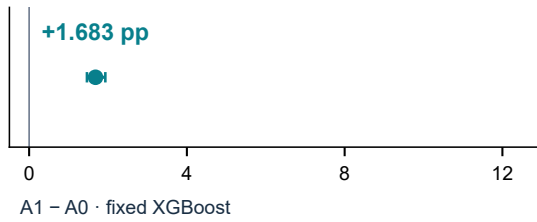
TABLE 2 | Main formal results. F1 is the equal-weight mean of positive-class F1 across evaluation cells. Intervals summarise paired differences using three seed clusters; Gate D remains descriptive.

Gate / comparison	Baseline F1	Method F1	Difference (pp)	95% CI (pp)
A: strict clean; A1 vs A0; fixed XGBoost	0.9149	0.9317	+1.683	[1.466, 1.932]
B: held-out generator; Full MSTF vs LPcode original (MLP)	0.8915	0.9671	+7.563	[7.452, 7.627]
C: combined transformation; Full MSTF vs LPcode original (MLP)	0.8415	0.9531	+11.160	[10.476, 12.010]
D: held-out language; Full MSTF vs LPcode original (MLP)	0.8938	0.9652	+7.141	[7.043, 7.228]

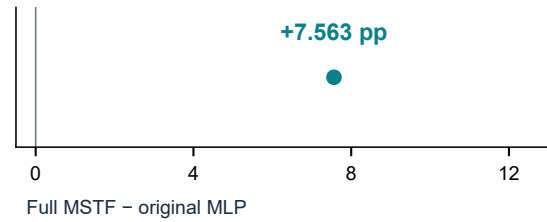
TABLE 3 | A0–A5 fixed-XGBoost ablation. Clean means use 60 cells per variant; held-out-generator means use 240. A5 is full MSTF. A4’s slightly higher clean score is retained.

Variant	Features	Representation	Dimensions	Clean F1	Held-out F1
A0	Original 10	Endpoints	20	0.9149	0.9089
A1	Original 10	Endpoints + signed difference	30	0.9317	0.9205
A2	Enhanced 28	Endpoints	56	0.9608	0.9587
A3	Enhanced 28	Signed difference only	28	0.9683	0.9641
A4	Enhanced 28	Endpoints + signed difference	84	0.9727	0.9671
A5	Enhanced 28	Endpoints + signed + relative	112	0.9725	0.9671

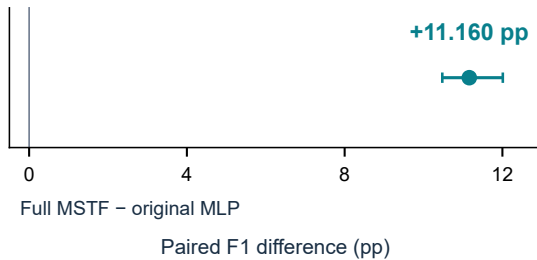
a Strict clean



b Held-out generator



c Combined transformation



d Held-out language

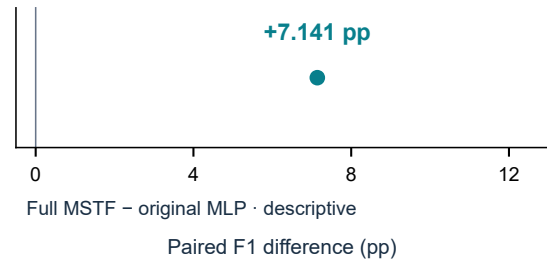


FIGURE 3 | Performance differences across the four evaluation gates. Points show mean paired F1 differences and lines show stored 95% seed-cluster CI ranges. Gate A compares A1 with A0 under fixed XGBoost; Gates B–D compare full MSTF with the original MLP. All intervals use three seed clusters, and Gate D is descriptive. Differences across panels cannot isolate a distribution-shift interaction because the comparator changes.

TABLE 4 | Held-out-generator results. Scores average four languages and 15 seed-fold units per language. Difference intervals resample the same three seed clusters; they are not independent per-generator significance tests.

Held-out generator	LPcode original	Full MSTF	Difference (pp)	95% CI (pp)
GPT-3.5	0.8978	0.9819	+8.412	[7.894, 8.768]
Gemini-Pro	0.8837	0.9592	+7.550	[7.196, 7.840]
WizardCoder-33B	0.9003	0.9687	+6.836	[6.463, 7.241]
DeepSeek-Coder-33B	0.8842	0.9587	+7.456	[7.236, 7.839]

7.4 | Combined transformations expose the larger robustness difference

Under the registered combined transformation, LPcode original reaches 0.8415 F1 and full MSTF reaches 0.9531. Their difference

is 11.160 points, with interval [10.476, 12.010] points. From their respective untransformed scores, the original model drops 6.311 points and MSTF drops 1.935 points. The relative reduction in absolute drop is 69.33%, calculated as one minus the MSTF drop divided by the original drop.

The individual conditions prevent the combined result from hiding weak or nearly inert perturbations. Formatting changes can modify code text without materially changing the inherited descriptor vector. Identifier renaming and combined edits test different sensitivities from comment injection. None of these transformations optimises against the trained detector, so the evidence should not be called unrestricted adversarial robustness.

7.5 | Language exclusion provides bounded transfer evidence

The equal-language means are 0.8938 for LPcode original and 0.9652 for MSTF. The average advantage is 7.141 points. Every held-out language and each seed-level aggregate favour MSTF, but only three seed clusters are present. Table 6 reports absolute scores and their descriptive seed dispersion.

This protocol tests omission of a language from fitting within one corpus. It does not test unseen repositories, languages outside the four evaluated here, or newly generated external examples. Shared task and collection regularities may still support transfer and limit its interpretation.

8 | Ablation, mechanism analysis and negative-pair sensitivity

8.1 | The controlled comparison matrix

Table 3 defines every variant and reports its absolute score. All variants use fixed XGBoost and identical train-test pair hashes. The design separates feature-family expansion, signed differences and the relative block, although it is not a complete factorial analysis of every possible interaction.

C1 is A1 minus A0, testing signed differences with the original features. C2 is A2 minus A0, testing enhanced descriptors without difference blocks. C3 is A4 minus A1, testing enhanced descriptors when signed differences are already present. C4 is A5 minus A4, testing the relative block. C5 is A5 minus A0, testing the complete representation change under fixed XGBoost. Exact contrast intervals appear in the supplementary evidence table.

8.2 | Enhanced endpoints and signed differences both contribute

C1 is +1.683 points in clean data and +1.153 points under generator exclusion. C2 is +4.592 and +4.976 points, respectively. Thus, enhanced endpoints alone account for a substantial controlled improvement. The data do not support saying that gains mainly come from transition rather than feature expansion.

C3 remains positive at +4.092 clean and +4.659 held-out points. Enhanced descriptors therefore remain useful when the original signed-difference representation is already available. A3 also performs strongly using enhanced signed differences alone. These comparisons support joint utility, but do not uniquely attribute

the complete system advantage to a particular family or a causal interaction.

8.3 | The relative block contributes little incrementally

C4 is -0.019 points in clean data and +0.007 points under generator exclusion. Its clean interval crosses zero. The positive held-out increment is extremely small, even though its stored bootstrap interval lies above zero. With three seed clusters, practical magnitude and dependence matter more than the sign of this narrow interval.

The finding argues against treating source-normalised differences as essential. It also prevents the full-model label from implying that A5 dominates A4. The frozen specification is retained for transparency; no retraining or outcome-driven feature removal was performed during manuscript revision.

8.4 | Descriptive rankings are not independent feature effects

The existing mechanism analysis describes fitted models on saved reconstructed splits. It includes 60 clean, 240 held-out-generator, 60 combined-transformation and 12 held-out-language evaluations. Grouped permutation importance ranks the relative structural/syntax family highest in the first three environments. The held-out-language environment instead favours the signed-difference original-style family. Top-ten permutation-ranking Jacard overlaps range from approximately 0.43 to 0.67.

These rankings describe how a fitted model uses available coordinates. They do not estimate the value of adding a feature block and refitting the model. Correlated endpoint and difference blocks can substitute for one another. A fitted model may rely on a relative coordinate even when a refitted model performs similarly without it. This distinction reconciles a high relative-block ranking with the near-null C4 result and is consistent with model-dependent variable importance [21].

8.5 | Negative-pair sensitivity tests the matched transition effect

The existing sensitivity study keeps positives, feature extraction, classifier configuration, seeds, folds and isolation constraints fixed. N0 reuses the frozen cross-component cyclic derangement. N1 selects deterministic random eligible negatives within language, generator and partition constraints. N2 selects the closest eligible candidate under a prediction-independent endpoint-style distance, with deterministic hash tie-breaking.

Table 7 reports the original ten-feature XGBoost comparison, A1 versus A0. The internal sensitivity artefact calls the transition method “mstf”, but it contains concatenation plus signed difference, not full 112-dimensional MSTF. Keeping this distinction explicit avoids overstating what the sensitivity experiment tests. Each language-specific difference is positive across the three constructions. Hard negatives lower both absolute scores, showing that the negative distribution materially affects task difficulty. The retained transition advantage is therefore not restricted to the

TABLE 5 | Deterministic transformation results. Drops are each method’s clean-minus-transformed mean F1, in points, on matched test pairs. Scores average four languages and three seeds with five folds; dispersion and paired intervals remain in the source data. Positive drops indicate degradation, and negative drops indicate improvement.

Transformation	LPcode original	Full MSTF	Difference (pp)	Drops: original / MSTF (pp)
Clean	0.9046	0.9725	+6.785	0.000 / 0.000
Comment removal	0.8957	0.9706	+7.488	0.890 / 0.187
Identifier rename	0.8534	0.9552	+10.177	5.124 / 1.732
Format normalization	0.9046	0.9724	+6.779	-0.001 / 0.004
Comment injection	0.9043	0.9713	+6.700	0.034 / 0.119
Combined	0.8415	0.9531	+11.160	6.311 / 1.935

TABLE 6 | Held-out-language transfer. Each row contains three seeded fits trained without the named language. Standard deviations describe those three fits; this is descriptive evidence, not a significance claim.

Held-out language	LPcode original: mean $\pm$ SD	MSTF: mean $\pm$ SD	Difference (pp)
C	0.8922 $\pm$ 0.0046	0.9656 $\pm$ 0.0010	+7.339
C++	0.8976 $\pm$ 0.0034	0.9723 $\pm$ 0.0009	+7.475
Java	0.8799 $\pm$ 0.0076	0.9584 $\pm$ 0.0021	+7.852
Python	0.9055 $\pm$ 0.0017	0.9645 $\pm$ 0.0014	+5.897

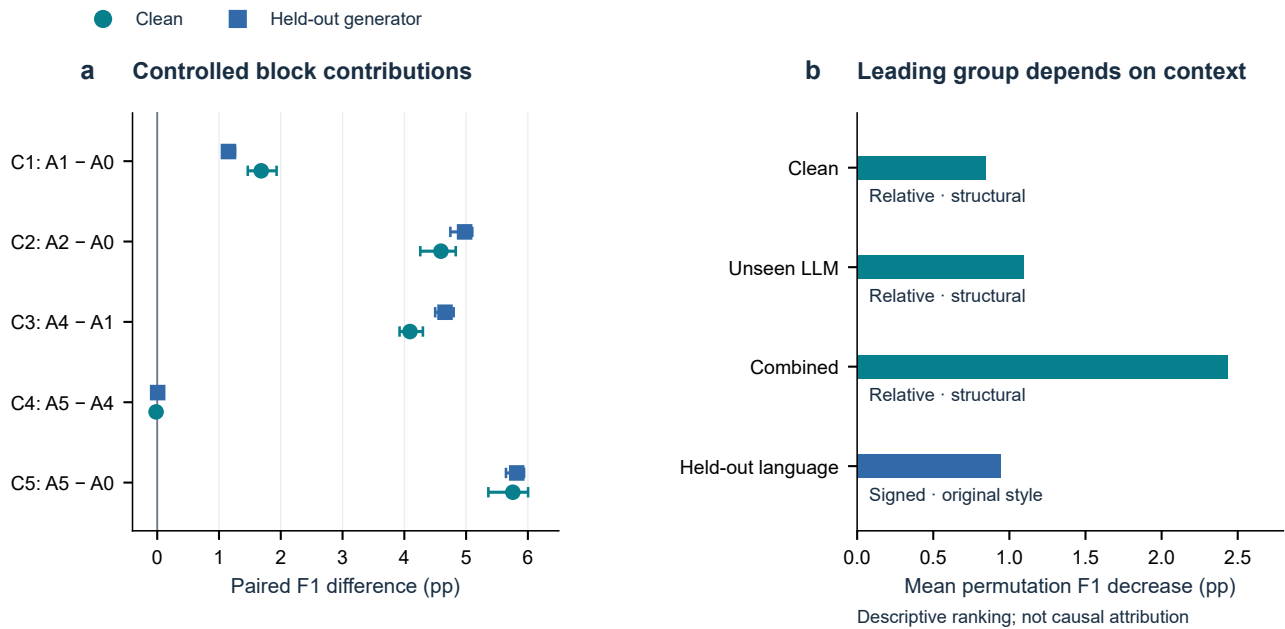


FIGURE 4 | Controlled contribution and descriptive model reliance. Left, C1–C5 compare clean and held-out-generator paired differences with stored 95% seed-cluster CI ranges. Right, the leading grouped permutation decrease differs across evaluation environments. Importance values are descriptive fitted-model summaries without causal interpretation; they do not replace the C4 refitting comparison.

TABLE 7 | Negative-pair sensitivity. Both methods use the inherited ten endpoint features and fixed XGBoost. Each mean uses 60 matched cells; intervals resample three seed clusters. N0 is reused frozen evidence, not a new experiment.

Negatives	A0 F1	A1 F1	Difference (pp)	95% CI (pp)
current	0.9149	0.9317	+1.683	[1.466, 1.932]
random	0.9119	0.9300	+1.805	[1.647, 1.896]
hard	0.8463	0.8824	+3.605	[3.431, 3.808]

original cyclic rule. It does not establish robustness to all semantically plausible unrelated candidates. Prediction-independent difficulty summaries and full language breakdowns accompany the evidence bundle.

9 | Discussion

9.1 | What the larger shifted-setting differences mean

Taken together, the results support combining richer endpoints with explicit source-relative differences under controlled paired evaluation. They extend LPcode’s stylistic approach [1] with a transparent representation and stricter isolation constraints. However, the larger shifted-setting headline differences use a different comparator from the clean headline. Their magnitudes cannot establish that the isolated signed-transition effect grows under distribution shift. In fact, C1 is smaller under generator exclusion than in clean data.

The whole-system pattern remains useful: full MSTF retains high performance relative to the original MLP under the evaluated shifts. A plausible interpretation is that richer measurements and an explicit change representation provide alternative predictive routes when some style cues are disrupted. The evidence does not uniquely separate that explanation from classifier capacity or corpus-specific regularities. Fixed-classifier contrasts constrain the interpretation but do not make it causal.

9.2 | Representation size is not the explanation by itself

The ablation does not support a simple contest between “more features” and “transition”. Enhanced endpoints produce substantial improvements without difference blocks, while signed differences add a smaller controlled benefit. The strong difference-only variant also cautions against equating predictive value with dimensionality. The near-null relative block shows that further concatenation need not improve performance.

This result suggests a practical design principle: make useful relational quantities accessible to the learner, but test each added block against a matched model. It does not require all normalised differences to be retained in future implementations. Any deployment-specific simplification would need validation on its own target distribution rather than relying on the present clean ranking alone.

9.3 | Failure boundaries and practical use

Figure 5 separates degradation under registered transformations from generalisation across generators and languages. The strongest negative-pair boundary is visible in N2: absolute detection becomes harder when unrelated candidates resemble the source in style. Such pairs challenge the central measurement strategy because stylistic proximity does not identify provenance uniquely. Conversely, a genuine paraphrase with extensive stylistic change can resemble an unrelated pair.

DeepSeek-Coder has the lowest full-MSTF mean among held-out generators, while Java has the lowest mean under language exclusion. The combined transformation produces the largest aggregate MSTF degradation, followed by identifier renaming. These are descriptive rankings within their respective protocols, not evidence that one language or generator is intrinsically harder in general. They also differ from ranking the largest advantage over the original comparator.

The available aggregate evidence supports environment-level failure analysis, but not an audited catalogue of individual false-positive and false-negative code pairs. We therefore do not invent representative errors or infer specific edit causes from feature rankings. A future case-level analysis would require preserved predictions, pair identifiers and source inspection. Semantic-level rewriting, repeated paraphrasing and cross-model paraphrase chains remain untested rather than demonstrated failures or successes.

Potential use is limited to settings where a meaningful human source is already available. The output may assist review of a proposed source–candidate relationship, but it cannot establish authorship, intent or misconduct. High scores under balanced constructed pairs do not imply calibrated risk under natural prevalence. Human verification and independent provenance evidence remain necessary before consequential interpretation.

10 | Threats to validity

10.1 | Construct validity

The positive label records a corpus provenance relationship, and negatives are constructed non-matches. These labels are not equivalent to authorship, semantic equivalence or deceptive intent. Handcrafted style measurements cover only selected observable properties. Parsing success and deterministic transformation execution also do not establish preserved runtime behaviour. These limits constrain the meaning of the classification target, even when prediction is accurate.

10.2 | Internal validity

Dual-endpoint isolation, exact-content components and common pair hashes address specific leakage and comparison risks. They do not remove approximate clones, shared task templates or implementation-specific regularities that lack exact hash equality. The ten inherited measurements retain heuristic extraction limitations. Parser fallback may also introduce systematic language-dependent behaviour. Matched ablations reduce classifier and pair confounding, while whole-system comparisons intentionally retain both descriptor and classifier differences.

10.3 | External validity

All formal results come from one LPcode corpus, four recorded generators and four languages. No independent external benchmark or newest-generation model evaluation is included. CodeMirage provides a relevant independent benchmark [3], but its existence should not be mistaken for validation of MSTF on it. Deterministic transformations do not cover semantic-level adversarial edits, multi-round rewrites or cross-model paraphrase chains. Generalisation claims therefore stop at the evaluated conditions.

10.4 | Statistical validity

Three seed clusters limit uncertainty estimation in every gate, not only Gate D. Folds share training data, and repeated seeds

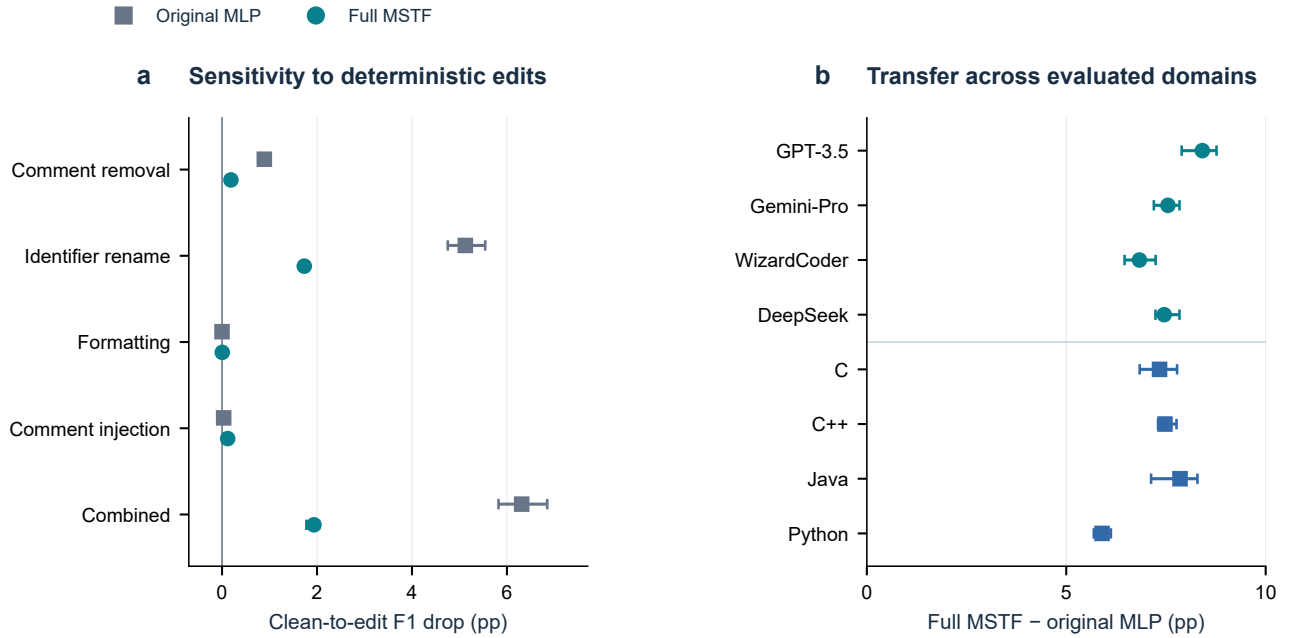


FIGURE 5 | Robustness and generalisation boundary. Left, clean-to-transformation drops for the original MLP and full MSTF, with stored seed-cluster intervals. Right, per-generator and per-language paired advantages, retaining all evaluated conditions. Generator and language points use their respective registered protocols; language evidence is descriptive. The figure covers deterministic transformations, not adaptive or semantic-level attacks.

reuse the corpus. Bootstrap intervals quantify limited experimental variation rather than uncertainty across all possible repositories, generators or developers. No fold-level independence assumption or population-level significance claim is made. Small relative-block increments should not be promoted into practical importance because a stored interval is narrowly positive.

10.5 | Reproducibility threats

Frozen ledgers, content hashes, code and pinned environments are publicly deposited. Public access permits independent inspection, but does not establish that a full training replication has been completed on another machine. Open-reuse licensing for original contributions still requires author confirmation; upstream rights remain separate. GitHub commit pinning is not a substitute for independent long-term archival preservation. These access and preservation limits do not change the local arithmetic.

11 | Conclusion

We studied LLM-paraphrased code as a leakage-sensitive paired provenance problem. Exact-content and dual-endpoint isolation support controlled comparison of endpoint and transition representations. Signed differences yield a modest strict-clean benefit under matched XGBoost. Full MSTF has larger advantages over the retained original MLP in the evaluated generator, transformation and language settings, with descriptive three-seed language evidence.

Controlled ablations show that enhanced endpoint descriptors and signed differences both contribute. The additional relative-difference block adds little incremental benefit, and its negative clean result remains part of the evidence. Hard-negative sensitivity further shows that absolute performance depends on the non-match distribution. These findings support a bounded paired-representation design principle within the evaluated corpus and protocols, not a universal detector or a method for assigning individual authorship.

DECLARATIONS

11.1 | Funding

No funding was received for this study.

11.2 | Conflict of interest

The authors declare no conflicts of interest.

11.3 | Data availability

The run-level records, frozen configurations and manifests, ablation and negative-pair summaries, and figure source data are publicly available in the commit-pinned research deposit [22]. Raw Lpcode snippets are obtained from <https://github.com/Shinwoo-Park/Lpcode> at commit b3660c8262ae57e14498528119607ee673d4257a; the deposit provides acquisition instructions and checksums rather than redistributing that corpus [1]. Public access was verified on 5

September 2026. No archival DOI is claimed. Open-reuse licensing for the authors' contributions remains subject to author confirmation; third-party rights are not superseded.

11.4 | Code availability

The complete research implementation, tests, pinned dependencies and reproduction commands are available in the same deposit [22]. The public release retains the frozen training and feature implementations, with documented packaging and path-resolution adaptations. An independent environment passed all 423 research tests and the saved-evidence integrity audit. These checks did not rerun the formal experiments and do not constitute an independent full training replication.

References

1. Park, Shinwoo; Jin, Hyundong; Cha, Jeong-won; Han, Yo-Sub. Detection of LLM-Paraphrased Code and Identification of the Responsible LLM Using Coding Style Features. arXiv:2502.17749; 2025. <https://arxiv.org/abs/2502.17749>
2. Arp, Daniel; Quiring, Erwin; Pendlebury, Feargus; Warnecke, Alexander; Pierazzi, Fabio; Wressnegger, Christian; Cavallaro, Lorenzo; Rieck, Konrad. Dos and Don'ts of Machine Learning in Computer Security. 31st USENIX Security Symposium, pp. 3971–3988; 2022. <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>
3. Guo, Hanxi; Cheng, Siyuan; Zhang, Kaiyuan; Shen, Guangyu; Zhang, Xiangyu. CodeMirage: A Multi-Lingual Benchmark for Detecting AI-Generated and Paraphrased Source Code from Production-Level LLMs. arXiv:2506.11059; 2025. <https://arxiv.org/abs/2506.11059>
4. Rahman, Musfiqur; Khatoonabadi, SayedHassan; Abdellatif, Ahmad; Shihab, Emad. Automatic Detection of LLM-Generated Code: A Comparative Case Study of Contemporary Models Across Function and Class Granularities. arXiv:2409.01382; 2024. <https://arxiv.org/abs/2409.01382>
5. Gurioli, Andrea; Gabbriellini, Maurizio; Zacchiroli, Stefano. Is This You, LLM? Recognizing AI-written Programs with Multilingual Code Stylometry. arXiv:2412.14611; 2024. <https://arxiv.org/abs/2412.14611>
6. Mitchell, Eric; Lee, Yoonho; Khazatsky, Alexander; Manning, Christopher D.; Finn, Chelsea. DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. arXiv:2301.11305; 2023. <https://arxiv.org/abs/2301.11305>
7. Bao, Guangsheng; Zhao, Yanbin; Teng, Zhiyang; Yang, Linyi; Zhang, Yue. Fast-DetectGPT: Efficient Zero-Shot Detection of Machine-Generated Text via Conditional Probability Curvature. arXiv:2310.05130; 2023. <https://arxiv.org/abs/2310.05130>
8. Sadasivan, Vinu Sankar; Kumar, Aounon; Balasubramanian, Sri-ram; Wang, Wenxiao; Feizi, Soheil. Can AI-Generated Text be Reliably Detected?. arXiv:2303.11156; 2023. <https://arxiv.org/abs/2303.11156>
9. Caliskan-Islam, Aylin; Harang, Richard; Liu, Andrew; Narayanan, Arvind; Voss, Clare; Yamaguchi, Fabian; Greenstadt, Rachel. De-anonymizing Programmers via Code Stylometry. 24th USENIX Security Symposium, pp. 255–270; 2015. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/caliskan-islam>
10. Bogomolov, Egor; Kovalenko, Vladimir; Rebryk, Yurii; Bacchelli, Alberto; Bryksin, Timofey. Authorship Attribution of Source Code: A Language-Agnostic Approach and Applicability in Software Engineering. 29th ACM Joint ESEC/FSE; 2021. <https://2021.esec-fse.org/details/fse-2021-papers/75/Authorship-Attribution-of-Source-Code-A-Language-Agnostic-Approach-and-Applcability>
11. Feng, Zhangyin; Guo, Daya; Tang, Duyu; Duan, Nan; Feng, Xiaocheng; Gong, Ming; Shou, Linjun; Qin, Bing; Liu, Ting; Jiang, Daxin; Zhou, Ming. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. arXiv:2002.08155; 2020. <https://arxiv.org/abs/2002.08155>
12. Guo, Daya; Ren, Shuo; Lu, Shuai; Feng, Zhangyin; Tang, Duyu; Liu, Shujie; Zhou, Long; Duan, Nan; Svyatkovskiy, Alexey; Fu, Shengyu; Tufano, Michele; Deng, Shao Kun; Clement, Colin; Drain, Dawn; Sundaresan, Neel; Yin, Jian; Jiang, Daxin; Zhou, Ming. GraphCodeBERT: Pre-training Code Representations with Data Flow. arXiv:2009.08366; 2020. <https://arxiv.org/abs/2009.08366>
13. Wang, Yue; Wang, Weishi; Joty, Shafiq; Hoi, Steven C. H.. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. arXiv:2109.00859; 2021. <https://arxiv.org/abs/2109.00859>
14. Guo, Daya; Lu, Shuai; Duan, Nan; Wang, Yanlin; Zhou, Ming; Yin, Jian. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. arXiv:2203.03850; 2022. <https://arxiv.org/abs/2203.03850>
15. Husain, Hamel; Wu, Ho-Hsiang; Gazit, Tiferet; Allamanis, Miltiadis; Brockschmidt, Marc. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. arXiv:1909.09436; 2019. <https://arxiv.org/abs/1909.09436>
16. Quiring, Erwin; Maier, Alwin; Rieck, Konrad. Misleading Authorship Attribution of Source Code using Adversarial Learning. 28th USENIX Security Symposium, pp. 479–496; 2019. <https://www.usenix.org/conference/usenixsecurity19/presentation/quiring>
17. Li, Zhen; Chen, Guenevere (Qian); Chen, Chen; Zou, Yayi; Xu, Shouhuai. RoPGen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation. 44th International Conference on Software Engineering; 2022. <https://doi.org/10.1145/3510003.3510181>
18. Chen, Tianqi; Guestrin, Carlos. XGBoost: A Scalable Tree Boosting System. arXiv:1603.02754; 2016. <https://arxiv.org/abs/1603.02754>
19. Pedregosa, Fabian; Varoquaux, Gaël; Gramfort, Alexandre; Michel, Vincent; Thirion, Bertrand; Grisel, Olivier; Blondel, Mathieu; Prettenhofer, Peter; Weiss, Ron; Dubourg, Vincent; Vanderplas, Jake; Passos, Alexandre; Cournapeau, David; Brucher, Matthieu; Perrot, Matthieu; Duchesnay, Édouard. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12(85):2825–2830; 2011. <https://www.jmlr.org/papers/v12/pedregosa11a.html>
20. Bengio, Yoshua; Grandvalet, Yves. No Unbiased Estimator of the Variance of K-Fold Cross-Validation. Journal of Machine Learning Research 5:1089–1105; 2004. <https://www.jmlr.org/papers/volume5/grandvalet04a/grandvalet04a.pdf>
21. Fisher, Aaron; Rudin, Cynthia; Dominici, Francesca. All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously. Journal of Machine Learning Research 20(177):1–81; 2019. <https://jmlr.org/papers/v20/18-760.html>
22. Huang, Jin; Li, Qiao; Gao, Qisen. LPcode / MSTF: leakage-safe paired provenance detection — software and numerical evidence. GitHub, research deposit, revision 1b6a5b9; 2026. <https://github.com/cdhuangjin/lpcode-mstf-leakage-safe-detection/tree/1b6a5b9f7f274b22a718b53219581d5f57a30792>